Hall Ticket: 2303A51360

Batch: 29

Assessment: 20.2

Task 1: Password Security Enhancement

Prompt:

Generate a Python script for user registration storing username and password.

Code:

task 20.2.py > main

```python
#Generate a Python script for user registration storing username and password.
# This script allows users to register by providing a username and password.
# The user data is stored in a dictionary for simplicity.
# Create an empty dictionary to store user data
import hashlib

# Dictionary to store user data
user_data = {}

# Function to hash passwords
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

# Register function
def register_user():
    username = input("Enter username: ")
    password = input("Enter password: ")

    if username in user_data:
        print("Username already exists!")
    else:
        user_data[username] = hash_password(password)
        print("User registered successfully!")

# Login function
def login_user():
    username = input("Enter username: ")
    password = input("Enter password: ")

    if username in user_data and user_data[username] == hash_password(password):
        print("Login successful!")
    else:
        print("Invalid username or password!")

# Main program
def main():
    while True:
        print("\n==== User System ====")
        print("1. Register")
        print("2. Login")
        print("3. Exit")
```

Output:

```
==== User System ====
1. Register
2. Login
3. Exit
Enter your choice: 1
Enter username: xyz
Enter password: abc@678
User registered successfully!

==== User System ====
1. Register
2. Login
3. Exit
Enter your choice: 2
Enter username: xyz
Enter password: abc@678
Login successful!
```

Significance:

This task shows the importance of securing user passwords by avoiding plain-text storage. Hashing ensures passwords are protected even if data is leaked, improving authentication security.

Task 2: Secure API Key Management

Prompt:

Generate a Python script that calls an API using an API key.

Code:

```python
#Generate a Python script that calls an API using an API key.
import requests

def call_api(api_url):
    try:
        response = requests.get(api_url)

        if response.status_code == 200:
            print("API call successful!")
            print(response.json())
        else:
            print("Error:", response.status_code)

    except Exception as e:
        print("Error:", e)

def main():
    api_url = input("Enter API URL: ")
    call_api(api_url)

if __name__ == "__main__":
    main()
```

Output:

```
Enter API URL: https://api.github.com
API call successful!
{'current_user_url': 'https://api.github.com/user', 'current_user_authorizations_html_url': 'https://github.com/settings/connections/applications{/client_id}', 'authorizations_url': 'https://ap
i.github.com/authorizations', 'code_search_url': 'https://api.github.com/search/code?q={query}{&page,per_page,sort,order}', 'commit_search_url': 'https://api.github.com/search/commits?q={query
{&page,per_page,sort,order}', 'emails_url': 'https://api.github.com/user/emails', 'emojis_url': 'https://api.github.com/emojis', 'events_url': 'https://api.github.com/events', 'feeds_url': 'htt
ps://api.github.com/feeds', 'followers_url': 'https://api.github.com/user/followers', 'following_url': 'https://api.github.com/user/following{/target}', 'gists_url': 'https://api.github.com/gis
ts{/gist_id}', 'hub_url': 'https://api.github.com/hub', 'issue_search_url': 'https://api.github.com/search/issues?q={query}{&page,per_page,sort,order}', 'issues_url': 'https://api.github.com/is
sues', 'keys_url': 'https://api.github.com/user/keys', 'label_search_url': 'https://api.github.com/search/labels?q={query}&repository_id={repository_id}{&page,per_page}', 'notifications_url':
https://api.github.com/notifications', 'organization_url': 'https://api.github.com/orgs/{org}', 'organization_repositories_url': 'https://api.github.com/orgs/{org}/repos{?type,page,per_page,sor
t}', 'organization_teams_url': 'https://api.github.com/orgs/{org}/teams', 'public_gists_url': 'https://api.github.com/gists/public', 'rate_limit_url': 'https://api.github.com/rate_limit', 'repo
sitory_url': 'https://api.github.com/repos/{owner}/{repo}', 'repository_search_url': 'https://api.github.com/search/repositories?q={query}{&page,per_page,sort,order}', 'current_user_repositori
s_url': 'https://api.github.com/user/repos{?type,page,per_page,sort}', 'starred_url': 'https://api.github.com/user/starred{/owner}{/repo}', 'starred_gists_url': 'https://api.github.com/gists/st
arred', 'topic_search_url': 'https://api.github.com/search/topics?q={query}{&page,per_page}', 'user_url': 'https://api.github.com/users/{user}', 'user_organizations_url': 'https://api.github.co
m/user/orgs', 'user_repositories_url': 'https://api.github.com/users/{user}/repos{?type,page,per_page,sort}', 'user_search_url': 'https://api.github.com/search/users?q={query}{&page,per_page,so
rt,order}'}
```

Significance:

This task highlights the risk of exposing API keys in source code. Using environment variable

Task 3: Weak Encryption Detection

Prompt:

Generate a simple encryption-decryption program."

Code:

```
#Generate a simple encryption-decryption program."
def encrypt(text, shift):
    encrypted = ""
    for char in text:
        if char.isalpha():
            shift_amount = shift % 26
            if char.islower():
                encrypted += chr((ord(char) - ord('a') + shift_amount) % 26 + ord('a'))
            else:
                encrypted += chr((ord(char) - ord('A') + shift_amount) % 26 + ord('A'))
        else:
            encrypted += char
    return encrypted

def decrypt(text, shift):
    decrypted = ""
    for char in text:
        if char.isalpha():
            shift_amount = shift % 26
            if char.islower():
                decrypted += chr((ord(char) - ord('a') - shift_amount) % 26 + ord('a'))
            else:
                decrypted += chr((ord(char) - ord('A') - shift_amount) % 26 + ord('A'))
        else:
            decrypted += char
    return decrypted

def main():
    choice = input("Do you want to (E)ncrypt or (D)ecrypt? ").upper()
    text = input("Enter the text: ")
    shift = int(input("Enter the shift value: "))

    if choice == 'E':
        result = encrypt(text, shift)
        print("Encrypted text:", result)
    elif choice == 'D':
        result = decrypt(text, shift)
        print("Decrypted text:", result)
    else:
        print("Invalid choice!")
if __name__ == "__main__":
    main()
```

Output:

```
Do you want to (E)ncrypt or (D)ecrypt? E
Enter the text: abcdef
Enter the shift value: 2
Encrypted text: cdefgh
```

Significance:

This task emphasizes replacing weak encryption with stronger algorithms. It ensures sensitive data remains secure and cannot be easily decoded by attackers.

Task 4: Real-Time Application: Security Review of AI-Generated Web Service

Prompt:

Generate a simple Python Flask web service with a login API that takes username and password from a POST request and validates them. Do not focus on security.

Code:

```python
#Generate a simple Python Flask web service with a login API that takes username and password from a POST request and validates them. Do not focus on security.
from flask import Flask, request, jsonify
import threading
import time
import requests

app = Flask(__name__)

# Sample user data
user_data = {
    "user1": "password1",
    "user2": "password2"
}

# Home route (for browser)
@app.route('/')
def home():
    return "Server is running!"

# Login API
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()

    username = data.get('username')
    password = data.get('password')

    if username in user_data and user_data[username] == password:
        return jsonify({"message": "Login successful!"})
    else:
        return jsonify({"message": "Invalid username or password!"})

# Function to test API automatically
def test_api():
    time.sleep(2)  # wait for server to start

    url = "http://127.0.0.1:5000/login"
    data = {
        "username": "user1",
        "password": "password1"
    }

    response = requests.post(url, json=data)
    print("\nAPI TEST RESULT:")
    print(response.json())

# Run everything
if __name__ == '__main__':
    # Run test in background
    threading.Thread(target=test_api).start()

    # Run Flask app
    app.run(debug=True)
```

Output:

```
API TEST RESULT:
{'message': 'Login successful!'}
API TEST RESULT:
{'message': 'Login successful!'}
127.0.0.1 - - [10/Apr/2026 19:32:50] "GET / HTTP/1.1" 200 -
```

Significance:

This task helps identify common vulnerabilities in web applications. It improves secure coding practices and prepares students for real-world security challenges.

Task 5: Exception Handling Improvement

Prompt:

Generate a simple Python program that takes two numbers as input from the user and performs division without using exception handling.

Code:

```python
#Generate a simple Python program that takes two numbers as input from the user and performs division without using exception handling.


def divide_numbers():
    try:
        # Taking user input
        a = int(input("Enter first number: "))
        b = int(input("Enter second number: "))

        # Performing division
        result = a / b

        print("Result:", result)

    except ValueError:
        print("Error: Please enter valid numbers only!")

    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")

    except Exception as e:
        print("Unexpected error:", e)

# Run program
if __name__ == "__main__":
    while True:
        divide_numbers()
        choice = input("\nDo you want to continue? (yes/no): ").lower()
        if choice != "yes":
            print("Program ended.")
            break
```

Output:

```
Enter first number: 3
Enter second number: 5
Result: 0.6
```

Significance:

This task improves program reliability by handling runtime errors. It prevents crashes and provides user-friendly error messages for better experience.